
AGENTICCADEIT: A STATEFUL, TOOL-MEDIATED AGENTIC APPROACH TO MULTIMODAL 3D CAD EDITING

Saptarshi Neil Sinha*

Fraunhofer IGD, Darmstadt, Germany
saptarshineilsinha@gmail.com

Mika Silvan Goshke

Fraunhofer IGD, Darmstadt, Germany
mika.silvan.goshke@igd.fraunhofer.de

Paul Julius Kühn

Fraunhofer IGD, Darmstadt, Germany
julius.kuehn@igd.fraunhofer.de

Arjan Kuijper

TU Darmstadt, Darmstadt, Germany
arjan.kuijper@igd.fraunhofer.de

Michael Weinmann

Delft University of Technology, Delft, Netherlands
m.weinmann@tudelft.nl

ABSTRACT

Computer-aided design is central to industrial manufacturing, and much of a designer’s daily work consists of editing existing models from multimodal requests involving speech, sketches, and model interaction. Existing neural CAD approaches focus predominantly on unconditional or text-conditioned generation. The neuralCAD-Edit approach formalizes expert multimodal editing requests, but its iterative baseline refines a complete CAD program across attempts, executing each attempt from the original model in a stateless CAD environment. Every attempt must therefore reconstruct the entire edit from scratch, so partially correct progress is discarded rather than accumulated, and the model can neither inspect the geometry it has just produced nor selectively revert a single faulty operation. We present AgenticCADedit, which turns editing into a sequence of small, verifiable actions on a persistent CAD state instead of a single regenerated program. Rather than emitting one complete program, it applies incremental code steps that each commit to the session, inspects the resulting faces and edges, renders highlighted selections to verify that the intended region was addressed, and reverts individual operations when it was not. Subsequent actions therefore build on the geometry produced by earlier ones. Our approach improves on all metrics for all three evaluated LLMs (open-weight: qwen3.6-27b, gemma4-31b; proprietary: gpt-5.6-luna), with the largest gains for the weakest baseline model, qwen3.6-27b, whose validity rises from 51.0% to 94.8% and acceptance from 1.6% to 12.0%. A token-cost analysis with gpt-5.6-luna further shows 66.7% fewer output tokens than neuralCAD-Edit, while 94.8% of input tokens are served from the prompt cache.

1 Introduction

Computer-aided design (CAD) is the medium in which industrial products are specified. Automotive bodies, machine components, and electronic housings alike exist as CAD geometry before a single part is produced, and original equipment manufacturers (OEMs) therefore rely on it across the entire product lifecycle. The boundary representation (B-Rep) [1] stored by these systems remains the reference representation from the first concept to the finally released part. Yet design is rarely a single act of creation. Parts are refined to meet tolerances, corrected after review, adapted to supplier constraints, and updated whenever a specification changes. Editing an existing model therefore constitutes a substantial part of day-to-day design work. Each such edit carries a communication cost. A reviewer describes a

*Corresponding author.

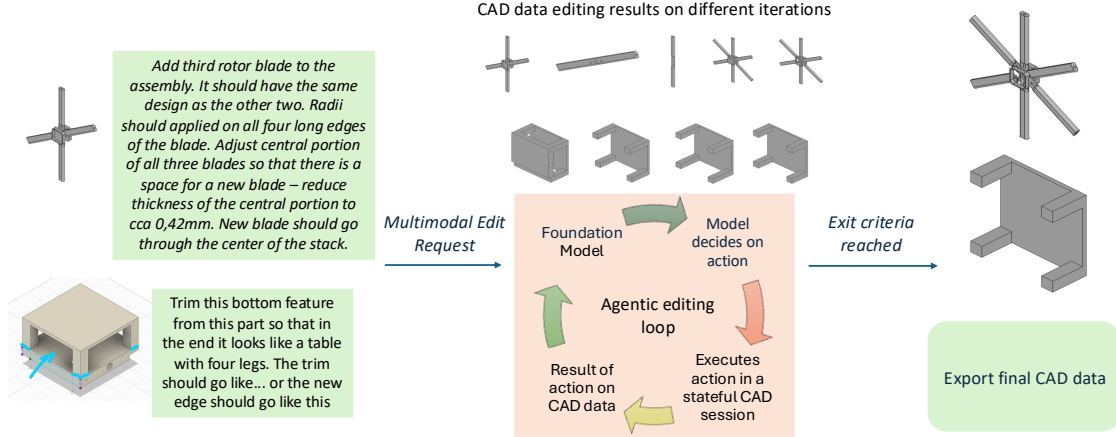


Figure 1: **Overview of AgenticCADedit.** Given a multimodal edit request and input CAD data, a foundation model incrementally edits a persistent CAD state. It can execute code, inspect geometry, render highlighted entities, and undo edits. Once the model declares completion, the result is exported.

change, a designer interprets it, opens the model, locates the relevant faces and edges in the underlying mesh, performs the modification, and repeats the process whenever the intent was only partially understood. Since time to market is a decisive factor in industrial competition, the latency of this exchange is an economic quantity rather than merely a matter of convenience. Automating every part of this process can directly shorten design iterations. However, the difficulty is that engineers do not communicate design changes through text alone. They speak while rotating the model, point at the face they mean, sketch contours on top of the geometry, and refer back to previous actions, so the instruction is multimodal and interleaved in time [2]. Moreover, editing is inherently cumulative: an operation changes the current geometry, subsequent operations act on the result, and an incorrect intermediate step may need to be inspected, revised, or reverted. A system for realistic CAD editing must therefore recover intent across modalities while interacting with an evolving model that it did not author.

Recent learning-based approaches have progressed from direct B-Rep synthesis [3, 4] to models that generate construction sequences [5, 6] and, more recently, executable CAD scripts [7, 8]. Editing has received substantially less attention. Existing methods remain predominantly text-conditioned, either infilling command sequences [9] or manipulating semantic latent representations [10]. neuralCAD-Edit [2] is, to our knowledge, the only approach that moves beyond text for editing existing 3D solids, recording professional designers speaking, pointing, and drawing while manipulating a model. The neuralCAD-Edit baseline provides feedback across attempts but no persistent editing state. Each iteration emits a complete CadQuery [11] function executed from the original model and hence, its rendering and output inform the next attempt, but its geometric effects do not persist. Editing therefore corresponds to repeated *full-program refinement*, i.e., the model reconstructs the complete edit from the original geometry instead of accumulating operations on an evolving model. In a stateful CAD environment, the model under construction persists across operations, so each executed step updates it and later steps build on the resulting geometry instead of re-deriving it. Prior agentic systems have explored this setting, including CAD-Assistant [12] and Embodied CAD [13]. However, these systems address different problem settings and tool spaces. CAD-Assistant primarily targets tasks such as sketch reasoning, autoconstraining, and parameterization, while Embodied CAD constructs assemblies from structured specifications through a fixed skill library. Neither studies stateful interaction as the editing paradigm for general multimodal modifications of existing 3D solids under a reference-based editing benchmark. This leaves open whether persistent, tool-mediated interaction benefits multimodal CAD editing. We investigate this question with **AgenticCADedit**, our stateful, tool-mediated approach to multimodal 3D CAD editing. To our knowledge, it is the first approach to combine video-recorded multimodal edit requests with stateful, tool-mediated editing of existing 3D CAD models. An LLM operates inside a persistent CadQuery [11] session exposed through a Model Context Protocol (MCP) server, applying incremental code steps rather than regenerating the complete edit. It can inspect faces and edges numerically, render highlighted selections, and undo unsuccessful operations. These tools form a perception–decision–action–verification loop in which the model chooses whether to inspect, verify, edit, revise, or terminate. State-changing calls modify the same session, so later actions operate on geometry and variables produced by earlier ones. Furthermore, we evaluate AgenticCADedit on the benchmark provided by neuralCAD-Edit [2] without modifying the benchmark or its evaluation protocol. This allows us to compare stateful incremental editing directly against the benchmark’s iterative but stateless

full-program refinement baseline across both open-weight and proprietary foundation models, while additionally measuring the token cost of the resulting sessions. Our contributions are:

- We introduce multimodal editing of existing 3D solids as *stateful incremental interaction* rather than repeated full-program refinement. Each successful edit modifies a persistent CAD state on which subsequent actions operate, avoiding regeneration of the complete edit from the original model at every iteration.
- We design an editing-oriented MCP tool interface around a persistent CadQuery session that combines incremental code execution with numerical geometry inspection, face and edge enumeration, rendered verification of selected entities, and explicit state management through undo operations.
- We isolate the effect of this interaction paradigm on neuralCAD-Edit [2], keeping the benchmark and evaluation protocol fixed so that only the interaction paradigm differs from the stateless script-refinement baseline. Our approach improves all metrics for every evaluated open-weight and proprietary model while emitting fewer output tokens, with the largest gains for qwen3.6-27b.

Table 1: MCP Tools available to the LLM during the agentic editing loop.

Tool	Purpose	Returns
<i>Edit</i>		
run_step	Executes CadQuery code in the persistent session	inspect summary and program output, or error message with traceback
<i>Query</i>		
inspect	Basic properties of the current model	Face and edge counts, axis-aligned bounding box
list_faces	Enumerates faces, sorted by area	Surface type, area, center, outward normal per face; total count
list_edges	Enumerates edges, sorted by length	Edge type, length, center, radius for circles; total count
<i>Render</i>		
render_views	Renders selected viewpoints, optionally highlighting faces or edges	One image per requested view
<i>State</i>		
export_cad	Exports the current CAD	Confirmation and path to the exported file
undo	Reverts the last successful edit	Updated model summary
reset	Clears the complete session and edit history, returning to an empty session	Confirmation that the session was reset
<i>Virtual tool</i>		
finish	Signals that the LLM considers the edit complete	Terminates the agentic loop after the current batch of tool calls

2 Related works

2.1 Neural CAD generation and representation

In industrial domains such as manufacturing and mobility, 3D data is predominantly represented in CAD systems as a boundary representation (B-Rep) [1, 14]. A B-Rep describes a solid through surface geometry and explicit topology, exposing faces, edges, and vertices as addressable entities that support direct geometric editing. In contrast, voxels, point clouds, meshes, radiance fields, and splats [15, 16, 17, 18, 19] represent shape through discrete samples, polygonal approximations, or learned fields and do not provide the same CAD-semantic structure for parametric editing. Consequently, methods developed for these representations generally do not produce editable B-Reps [20, 21, 22, 23], making CAD generation and editing a distinct research area.

CAD benchmark datasets Several benchmark datasets have been created for this field to support learning-based approaches to CAD generation and manipulation. At the largest scale, one million B-Reps are collected in ABC [24], which has enabled substantial progress in geometric deep learning but keeps only the final geometry and discards the construction history, while millions of sketch-and-constraint graphs are provided by SketchGraphs [25] without extending to 3D solid operations. Capturing design intent requires the construction timeline, which only the Fusion 360 Gallery [26, 27] and DeepCAD [5] expose, the former with human-designed sequences at a small scale of 8.6k models and the latter with 178k sequences parsed from the public Onshape documents behind ABC. Both remain restricted to

sketch-and-extrude operations, and operation types and step order are annotated in CC3D-Ops [28] with per-face labels rather than replayable programs.

Generative CAD approaches In generative CAD two main approaches exist, and they differ in the representation they produce. The first generates B-Reps directly, through diffusion [29, 3, 30, 31], autoregressive decoding [4, 32, 33] and single-view reconstruction [34], supported by geometry encoders such as UV-Net [35], which yields valid solids but no construction history and usually only a single body. The second models the construction sequence itself, as a token stream [5, 36, 37, 38, 39] or through hierarchical codebooks [40, 41], conditioned on point clouds [42, 43, 44], or on text, images and structural masks [6, 45, 46, 47, 48, 49, 50, 51], with Text2CAD [6] showing that language can drive parametric construction while inheriting DeepCAD’s narrow operation vocabulary. A more recent line of work sidesteps this fixed command set by emitting general CAD scripts, as in CAD-Recode [7], which reconstructs executable CadQuery [11] code from point clouds and is trained on procedurally generated sketch-extrude programs, CAD-Coder [8], which generates such code from images, and Text-to-CadQuery [52], which generates it from text descriptions. We follow this script view and operate on executable CadQuery code, since it covers the operations a real edit requires, such as fillets, chamfers, shells and Booleans, and stays directly interpretable.

2.2 Agentic CAD modelling

Agentic AI for CAD modelling has become an active area of research, with a growing body of work placing a foundation model in a loop with an executable CAD environment rather than decoding a design in one pass. Some approaches write CAD scripts, run them and repair them from the resulting execution errors [53, 48, 54, 55, 56, 57], while others directly operate CAD-specific tools through vision-language models (VLMs) [12, 13, 58, 59]. A common observation is that execution feedback alone is insufficient for geometric tasks, since the reported failures are locally valid but globally incoherent, which has motivated rendered feedback for iterative refinement [60, 61, 45, 62] and VLM-based raters for text-to-3D alignment [63, 6]. IterCAD [62] embeds an editing task in a multi-turn sandbox. Its edit instructions are text-only and its source programs are synthetic, obtained by degrading correct scripts. Its multimodal input consists of the engineering drawings that define the target part. Prior agentic CAD systems differ in how they carry information across turns. IterCAD [62] and the neuralCAD-Edit [2] baseline re-provide the interaction history but re-execute a complete program from the original model in every attempt. CAD-Assistant [12] and Embodied CAD [13] maintain a persistent session, yet address different tasks, namely question answering, autoconstraining and sketch parameterization in the former case and assembly construction from structured specifications in the latter, without comparing the resulting geometry to a reference model. We apply persistent, tool-mediated interaction to multimodal edits of existing solids under a reference-based benchmark, where the agent modifies one CAD state incrementally, verifies intermediate geometry visually and can undo an unsuccessful operation, so later actions build on earlier edits. A second limitation is methodological. Each agent is compared against a weaker or differently trained system, and several are fine-tuned for the task [62, 58, 13], so the contribution of the interface cannot be separated from the contribution of the training. We take inspiration from these systems and address multimodal edits of an existing CAD model, holding the model, the sampling parameters and the benchmark fixed so that only the interface varies.

2.3 Multimodal CAD editing and benchmarks

CAD editing has been far less explored than generation, and the conditioning signals that are available remain narrow. Text-based approaches either treat editing as a locate-then-infill problem over command sequences, evaluated on a synthetic benchmark of single components [9], modify a semantic B-Rep latent space with an LLM [10], or edit through a domain-specific language whose named entities replace coordinates [64]. Related resources outside CAD are similarly restricted, since they rely on text-only requests [22], on a rendered goal image that presupposes the completed edit [65], or on synthetic sketch conditioning [23]. Two benchmarks move beyond text. mrCAD [66] collects refinement instructions that combine text and drawings, and reports that state-of-the-art VLMs follow generation instructions considerably better than refinement instructions, but it is restricted to a 2D CAD environment. neuralCAD-Edit [2] is, to our knowledge, the only approach that moves beyond text for 3D solids, collecting requests in which professional designers speak, point and draw while manipulating the model, together with reference edits for single bodies and assemblies. In this setting the limits of stateless refinement become most visible when compared against our approach. We evaluate on this benchmark and reuse its evaluation protocol unchanged, so any difference follows from the interaction paradigm.

3 Methodology

We introduce an agentic approach for editing CAD data from multimodal edit requests. Instead of asking a large language model (LLM) to produce one complete CAD program per iteration, our agentic CAD editing arm, shown in

Table 2: Mean over all tasks ($N = 192$). Failed or invalid edits are scored as zero. The LLM model is given in parentheses.

edit_user	validity (%) $\uparrow$	instruction $\uparrow$	quality $\uparrow$	acceptance (%) $\uparrow$
Human baseline	100.00	0.75	0.75	82.30
neuralCAD-Edit [2] (qwen3.6-27b)	51.00	0.17	0.17	1.60
Ours (qwen3.6-27b)	94.80	0.32	0.39	12.00
neuralCAD-Edit [2] (gemma4-31b)	40.10	0.19	0.15	2.60
Ours (gemma4-31b)	57.30	0.23	0.21	6.80
neuralCAD-Edit [2] (gpt-5.6-luna)	96.40	0.57	0.57	50.00
Ours (gpt-5.6-luna)	99.00	0.59	0.60	53.60

Table 3: Median over successful tasks. `n_valid` is the number of tasks that produced a valid, gradable CAD output file. Valid subsets differ across rows; Tab. 4 provides the pairwise comparison.

edit_user	n_valid $\uparrow$	chamfer_dist $\downarrow$	iou $\uparrow$	DINO-sim $\uparrow$
Human baseline	192.00	2.65	0.95	0.99
neuralCAD-Edit [2] (qwen3.6-27b)	98.00	4.26	0.64	0.53
Ours (qwen3.6-27b)	182.00	3.72	0.71	0.57
neuralCAD-Edit [2] (gemma4-31b)	77.00	9.41	0.54	0.62
Ours (gemma4-31b)	110.00	8.80	0.55	0.65
neuralCAD-Edit [2] (gpt-5.6-luna)	185.00	3.15	0.83	0.63
Ours (gpt-5.6-luna)	190.00	2.98	0.87	0.64

Fig. 1, gives the model direct and incremental control over a CAD session through a Model Context Protocol (MCP) interface. The LLM manipulates geometry by tool calls which are carried out by a harness, i.e. the program that drives the agentic loop and mediates between the LLM and a persistent CAD session. Our agentic CAD editing approach therefore becomes a sequence of small and verifiable actions. The following subsections describe the context given to the LLM, the execution environment, the agentic loop, the exposed tools, the model interface and the exit criteria.

3.1 LLM context

Before the agentic loop starts, the LLM receives a prompt consisting of a conversation instruction, the schemas of the tools exposed by the MCP server, and the task information. The conversation instruction states that the LLM works inside a persistent session of a parametric CAD scripting framework, CadQuery [11] in our case, in which the framework is already imported, the model to be edited is already loaded, variables persist across steps, the evolving geometry is always bound to a single result variable, and the task is solved through tool calls. It further advises the LLM to inspect and render the model before editing, to keep code steps small, to visually confirm the relevant faces and edges before operating on them, to verify the result after each modification, and to report completion only once the model matches the request. Following neuralCADEdit [2], we support the same multimodal edit requests, namely text passed directly as a string and videos in which the designer speaks while pointing at, rotating and drawing on the model.

3.2 Agentic CAD editing loop

The main agentic loop (see Figure 1) drives the interaction between the LLM and the CAD session. In each iteration, the LLM is called with the complete conversation history and the available tool schemas. If the LLM requests one or more tools, the harness executes them in the order they were requested and appends their results to the conversation, so that the LLM observes the effect of its own actions in the next iteration. Several tools can be requested in a single turn, which allows the LLM to combine an edit with a subsequent inspection or rendering without spending an additional iteration on it. Every tool call is bounded by a time limit, so that a single long-running operation cannot stall the whole run. The loop continues with the updated conversation until one of the exit criteria is met, at which point the current CAD model is exported for evaluation.

3.2.1 Stateful execution environment

The harness launches the MCP server as a subprocess and communicates over standard input and output. Its persistent namespace imports CadQuery once and retains LLM-defined variables across calls, allowing helper variables to be reused instead of regenerating a complete program. The current geometry is stored in `result`. Before every code execution the server pushes the session state onto a history stack, so that a failed step restores the previous state and never leaves the session inconsistent, while the undo tool reverts an edit that executed successfully but produced an unintended result.

3.2.2 MCP tools

The MCP tools define the action space of the LLM and they cover four kinds of interaction with the session, namely executing code, querying the geometry numerically, rendering it visually, and managing the session state. The numerical tools are needed because the LLM cannot reliably infer dimensions or selector strings from images alone, and the rendering tool is needed because numerical output does not reveal whether the intended region was actually selected. We present the overview of each tool in Table 1. The core editing tool is **run_step**, which executes CadQuery [11] code in the persistent session so that variables and progress carry over between calls, with the current model always stored in `result` and the previous state restored on error. The remaining tools let the LLM query the geometry numerically,

Table 4: Pairwise comparison of mean scores. Both rows in a pair use the same LLM (given in parentheses), the same evaluation protocol, and are evaluated on the identical set of tasks, so any difference comes from the interaction paradigm alone.

edit_user	chamfer dist ↓	iou ↑	Dino-sim ↑	instruction ↑	quality ↑	acceptance (%) ↑	n_tasks
neuralCAD-Edit [2] (qwen3.6-27b)	66.60	0.56	0.59	0.18	0.34	3.20	95.00
Ours (qwen3.6-27b)	21.84	0.60	0.61	0.36	0.42	15.80	95.00
neuralCAD-Edit [2] (gemma4-31b)	58.47	0.49	0.60	0.23	0.35	5.60	71.00
Ours (gemma4-31b)	78.3	0.57	0.61	0.28	0.36	9.90	71.00
neuralCAD-Edit [2] (gpt-5.6-luna)	24.38	0.66	0.64	0.58	0.60	52.20	184.00
Ours (gpt-5.6-luna)	24.21	0.69	0.65	0.60	0.61	55.40	184.00

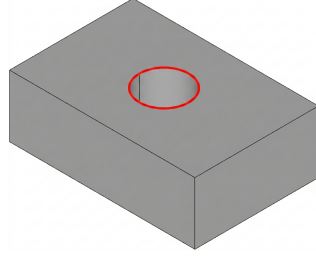


Figure 2: Two edges of the B-Rep selected, marked in red

verify visually that a selector matched the intended region as shown in Figure 2, manage the session state, and signal completion of the edit through **finish**, which terminates the agentic loop.

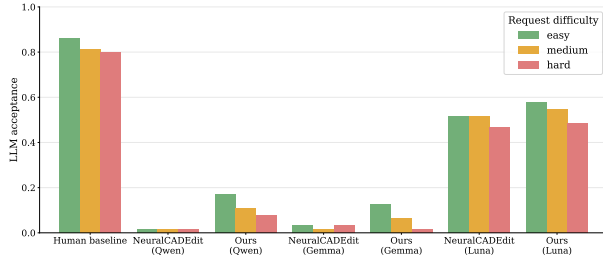


Figure 3: LLM acceptance grouped by request difficulty

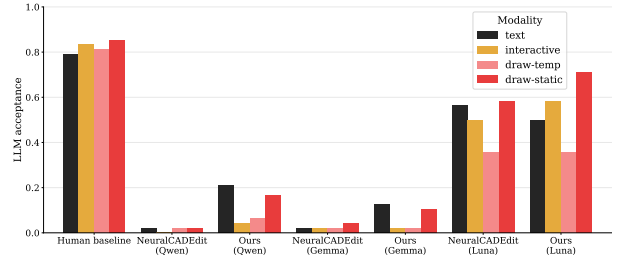


Figure 4: LLM acceptance grouped by input modality

3.2.3 LLM interface

We use the OpenAI tool-calling format so different models share one harness. Tool schemas accompany every request; model responses and textual tool results are appended unchanged with their call identifiers. Because tool messages cannot contain images, renders from a turn are collected in one additional user message. The harness records token counts, transcripts, and images, including for failed runs, enabling behavioral inspection and cost analysis.

3.2.4 Exit criteria

The loop ends in one of three ways. The intended case is that the LLM calls the **finish** tool, which signals that it considers the edit complete. The second case is that the LLM returns a response without any tool call, which is treated as an implicit end of the trajectory. The third case is that the iteration budget is exhausted, which bounds the cost of a run and prevents the LLM from looping indefinitely on a step it cannot solve. Both approaches use a maximum of 100 LLM iterations per task. In all three cases, the current value of **result** is exported to a CAD format supported by our framework and passed on to the evaluation, so that every run produces a model that can be compared against the reference.

4 Dataset

We evaluate on the neuralCAD-Edit benchmark [2], which provides 192 expert multimodal CAD editing requests. The requests span four input modalities: *text*, consisting of typed instructions without speech or direct model interaction;

interactive, in which designers speak while pointing at and manipulating the CAD model; *draw-temp*, which additionally includes temporary drawings that disappear after a short period; and *draw-static*, in which drawings remain visible throughout the request or until manually removed. The multimodal requests are recorded as videos, preserving the temporal alignment between speech, model interaction, viewpoint changes, and drawings. The benchmark further covers three difficulty levels and four model categories formed by the combinations of parametric versus non-parametric models and single-body versus assembly construction.

5 Implementation details

The open-weight models qwen3.6-27b [67] and gemma4-31b [68] were self-hosted on a GPU cluster equipped with four NVIDIA A100 GPUs (40 GB each). In contrast, gpt-5.6-luna [69] was accessed through its public API with the reasoning effort set to high.

6 Evaluation

We follow the baseline metrics: Chamfer distance, voxel IoU, DINOv2 similarity on isometric renders [70], validity, and VLM ratings for instruction following, model quality, and acceptance. Instruction and quality are rated from 1 to 7 and normalized to $[0, 1]$; acceptance requires both scores to be at least 5. Opus 5 medium, operated inside Claude Code, produces all ratings by inspecting and comparing rendered regions. Unlike the earlier VLM evaluators considered in neuralCAD-Edit [2], Opus 5 inside Claude Code closely reproduces the expert ratings on the human baseline, providing stronger evidence that its automatic ratings are meaningful for this benchmark. Our qualitative analysis uses renders and transcripts from easy, medium, and hard examples.

6.1 Comparison with Baseline Approaches

As the primary baseline for comparison, we use the iterative script-refinement baseline released with neuralCAD-Edit [2]. We evaluate the same multimodal requests with and without the stateful session for proprietary gpt-5.6-luna [69] and open-weight gemma4-31b [68] and qwen3.6-27b [67]. Using each model in both harnesses separates the interaction paradigm from the underlying model.

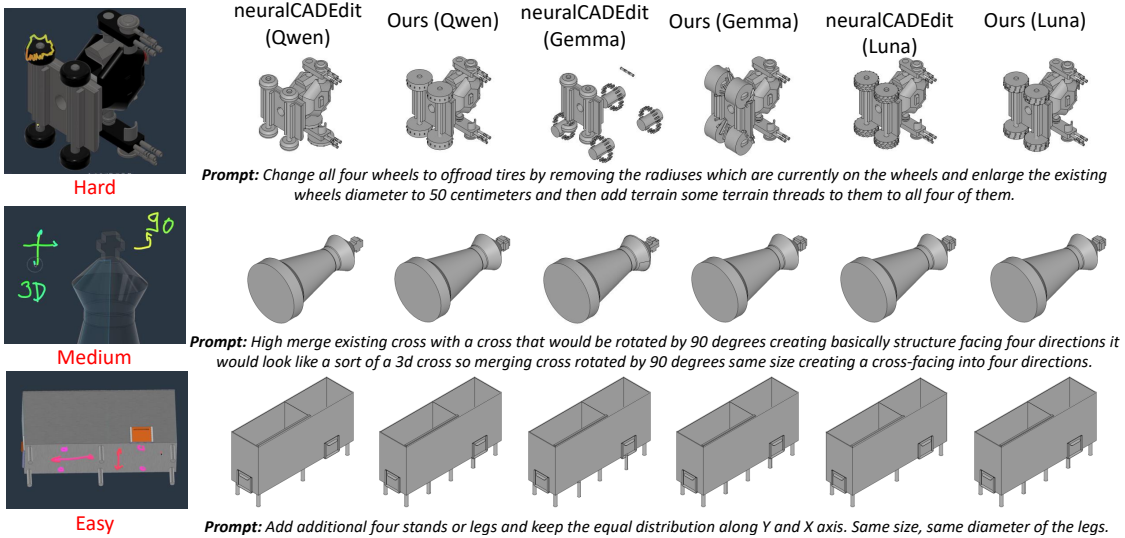


Figure 5: Qualitative comparison across request difficulty. Each row shows one editing request of easy, medium or hard difficulty, and the columns show the resulting edits of neuralCAD-Edit [2] and of our approach, each instantiated with qwen3.6-27b [67], gemma4-31b [68] and gpt-5.6-luna [69].

6.2 Quantitative results

Our approach improves over neuralCAD-Edit [2] for every LLM and on every metric of the mean evaluation over all 192 tasks in Table 2, where failed or invalid edits are scored as zero. The relative gains are largest for the lightweight

open weight models, since validity almost doubles and acceptance increases by more than a factor of seven for qwen3.6-27b [67], suggesting that the more limited open weight models benefit particularly strongly from the structured interaction provided by the agentic session, whereas for gpt-5.6-luna [69] the baseline is already strong and the margins are smaller.

Since using only mean conflates how often a system produces usable output with how good that output is, we also report (see Table 3) the number of valid outputs together with the median geometric scores over exactly those tasks. The number of valid outputs grows significantly for the open weight models, while the median Chamfer distance, IoU and DINO similarity improve by smaller margins, which suggests that our approach mainly converts previously broken attempts into usable edits and that the accuracy of an already usable edit depends more on the underlying model. Since these medians are computed over a different subset for each row, we additionally restrict (see Table 4) each pair to the tasks that both systems solved validly and holds model, iteration budget and protocol fixed, so any remaining difference comes from the interaction paradigm alone. Similar conclusions hold under this stricter setting, and the only exception is the mean Chamfer distance, which is sensitive to a few strongly displaced outputs and can therefore worsen even when all other metrics improve. Across all three views the human baseline stays ahead, in particular on acceptance, so there is clear scope for improvement.

We also evaluate acceptance in a more fine grained way. Fig. 3 groups the acceptance rate by request difficulty into easy, medium and hard requests, and Fig. 4 groups it by input modality into text, interactive, draw-temp and draw-static requests, in both cases for the human baseline and for all three pairs of systems. Acceptance generally decreases with request difficulty, most clearly for the human baseline and our agentic variants. Our approach yields substantial gains for qwen3.6-27b [67] and gpt-5.6-luna [69] across all difficulty levels and for gemma4-31b [68] on easy and medium requests, while hard requests remain particularly challenging for the open weight models. Interactive and draw-temp requests are particularly difficult for open-weight models, probably because cursor motion and transient annotations must be recovered from frame sequences. gpt-5.6-luna handles interaction substantially better, but draw-temp remains its weakest modality. Persistent draw-static annotations reduce this ambiguity and yield higher acceptance, especially for gpt-5.6-luna.

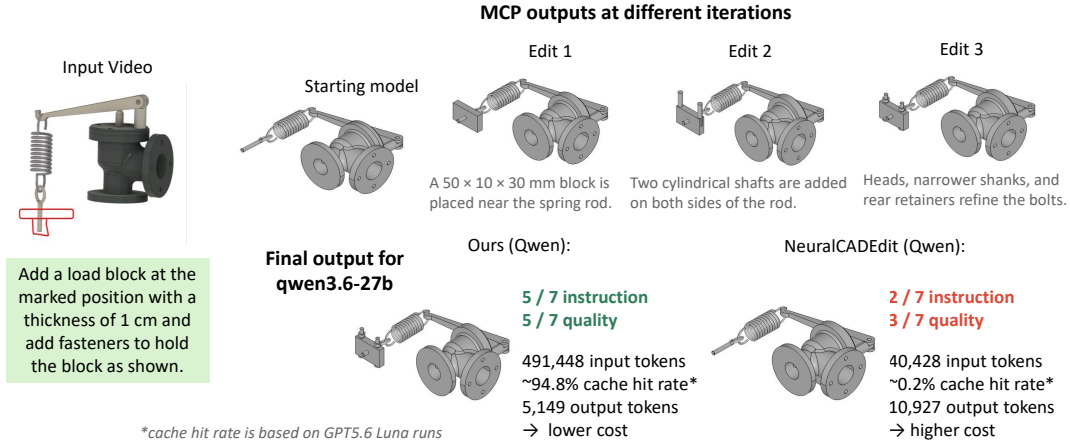


Figure 6: Ablation of the agentic session. Given the starting model and the multimodal edit instruction on the left, we show the intermediate outputs of our stateful agentic loop after each operation, together with the final output of the neuralCAD-Edit full-program refinement baseline [2]. Both systems use qwen3.6-27b [67], so the difference isolates the effect of operating incrementally on a persistent CAD state instead of regenerating a complete edit program across refinement attempts.

6.3 Qualitative results

We now qualitatively analyze the edits in Fig. 5 to complement the previous quantitative evaluation with visual results. For the easy request, which asks for four additional legs with equal distribution along the X and Y axes and the same size and diameter, our approach preserves both the leg dimensions and the symmetry of the placement for all three models. With qwen3.6-27b [67] the baseline either changes the diameter of the added legs or places them off the intended grid, and with gemma4-31b [68] it frequently fails to produce a valid body at all, whereas our approach recovers a usable and consistent result in both cases. With gpt-5.6-luna [69] both systems solve the request, so the difference reduces to the accuracy of the placement.

For the medium request, which asks to merge the existing cross with a copy rotated by ninety degrees so that the result faces four directions, the gap between the models becomes more visible. With qwen3.6-27b [67] the baseline typically leaves the original cross almost unchanged or adds a misaligned arm, while our approach produces the intended three dimensional cross. With gemma4-31b [68] the rotation is often applied to the wrong axis or only a partially merged body is created, and even our approach does not always close the symmetry completely, whereas with gpt-5.6-luna [69] the baseline occasionally duplicates the geometry without merging it and our approach yields a clean single body. The hard request combines removing wheel radiuses, enlarging wheel diameter, and adding treads to all four wheels. Open-weight models usually complete only the enlargement, while gpt-5.6-luna addresses more steps but leaves coarse treads. This mirrors the difficulty trend: the agentic session makes well-specified edits more reliable, while long multi-step requests remain challenging.

6.4 Ablation studies

The primary ablation of the agentic session is already contained in Table 2, Table 3 and Table 4. Both approaches use the same underlying LLM, but differ in how the CAD environment is exposed: the neuralCAD-Edit [2] baseline repeatedly refines a complete program that is executed in a stateless CAD environment, whereas our approach performs incremental tool-mediated edits on a persistent CAD state. The consistent improvements therefore quantify the contribution of the interaction paradigm.

Fig. 6 compares the final output of the full-program refinement baseline with the intermediate states produced by our agentic loop for an instruction to add a 1 cm-thick load block at the marked location and secure it with fasteners. The neuralCAD-Edit [2] baseline places an incorrectly sized and positioned block and produces incomplete fasteners. In contrast, our loop first adds the load block, then introduces the bolt shafts, and finally refines them with heads, shanks, and rear retainers. Each intermediate edit remains valid, and the final result better follows the requested geometry and attachment structure. We additionally apply the LLM-based instruction, quality and acceptance ratings described above to the example in Fig. 6. Both systems use qwen3.6-27b [67], so the comparison isolates the effect of the interaction paradigm. Our final output receives an instruction score of 5 out of 7 and a quality score of 5 out of 7, while the full-program refinement baseline reaches only 2 out of 7 and 3 out of 7.

6.5 Token cost

Across all 192 tasks with gpt-5.6-luna [69], our loop processes 301.37 million input tokens versus 99.39 million for neuralCAD-Edit [2], but 94.8% are served from the prompt cache compared with only 0.2% for the baseline. This high cache reuse follows from the append-only multi-turn session, whose successive requests share a largely unchanged prefix. Our loop also reduces output tokens from 5.39 million to 1.80 million (66.7%). Consequently, the recorded API cost is only \$11.00 for our loop versus \$26.30 for the baseline, a reduction of 58.2%, despite the higher nominal input-token volume.

7 Conclusion

We presented AgenticCADedit, an approach that reformulates multimodal CAD editing from stateless full-program refinement into stateful, tool-mediated interaction with an evolving CAD model. On the neuralCAD-Edit benchmark with 192 expert multimodal requests, our approach increases validity, instruction following, model quality and acceptance for every evaluated LLM, with the largest gains for open-weight models, where validity almost doubles and acceptance increases by more than a factor of seven for qwen3.6-27b. The pairwise comparison on identical task sets, with model, iteration budget and evaluation protocol held fixed, further isolates the effect of the interaction paradigm from that of the underlying foundation model. Rather than repeatedly reconstructing a complete edit from the original geometry, the model can act on, inspect, verify and revise a persistent geometric state. Our gpt-5.6-luna cost analysis further shows 66.7% fewer output tokens, while 94.8% of input tokens are served from the prompt cache.

The human baseline nevertheless remains ahead, and long composite requests and sketch-based intent remain only partially solved. To address this limitation, future work should introduce an explicit planning stage over the requested operations and tighter grounding of annotated video input to the corresponding geometric entities. Beyond this, a richer set of query and verification tools could further reduce the remaining gap to expert edits.

References

- [1] Kevin J. Weiler. *Topological Structures for Geometric Modeling*. PhD thesis, Rensselaer Polytechnic Institute, 1986.

- [2] Toby Perrett, Matthew Bourchard, and William McCarthy. neuralcad-edit: An expert benchmark for multimodal-instructed 3d cad model editing. In *arXiv preprint arXiv:2604.16170*, 2026.
- [3] Xiang Xu, Joseph G. Lambourne, Pradeep Kumar Jayaraman, Zhengqing Wang, Karl D. D. Willis, and Yasutaka Furukawa. BrepGen: A b-rep generative diffusion model with structured latent geometry. *ACM Transactions on Graphics (TOG)*, 43(4):1–14, 2024.
- [4] Pradeep Kumar Jayaraman, Joseph G. Lambourne, Nishkrit Desai, Karl D. D. Willis, Aditya Sanghi, and Nigel J. W. Morris. SolidGen: An autoregressive model for direct b-rep synthesis. *arXiv preprint arXiv:2203.13944*, 2022.
- [5] Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models. In *International Conference on Computer Vision (ICCV)*, pages 6772–6782, 2021.
- [6] Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2cad: Generating sequential cad models from beginner-to-expert level text prompts. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [7] Danila Rukhovich, Elona Dupont, Dimitrios Mallis, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-recode: Reverse engineering cad code from point clouds. In *International Conference on Computer Vision (ICCV)*, pages 9801–9811, 2025.
- [8] Anna C. Doris, Md Ferdous Alam, Amin Heyrani Nobari, and Faez Ahmed. Cad-coder: An open-source vision-language model for computer-aided design code generation. *Journal of Mechanical Design*, 148(7):071702, 2026.
- [9] Yu Yuan, Shizhao Sun, Qi Liu, and Jiang Bian. Cad-editor: A locate-then-infill framework with automated training data synthesis for text-based cad editing, 2025.
- [10] Yilin Liu, Niladri Shekhar Dutt, Changjian Li, and Niloy J. Mitra. B-repler: Language-guided editing of cad models, 2026.
- [11] CadQuery contributors. Cadquery, June 2026.
- [12] Dimitrios Mallis, Ahmet Serdar Karadeniz, Sebastian Cavada, Danila Rukhovich, Niki Foteinopoulou, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-assistant: Tool-augmented vllms as generic cad task solvers, 2025.
- [13] Fumin Liu, Haoyu Zhou, Fei Hao, and Lin Yang. Embodied cad: Solver-grounded llm agents for parametric b-rep assembly modeling, 2026.
- [14] Joseph G. Lambourne, Karl D.D. Willis, Pradeep Kumar Jayaraman, Aditya Sanghi, Peter Meltzer, and Hooman Shayani. Brepnet: A topological message passing system for solid models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12773–12782, June 2021.
- [15] Linqi Zhou, Yilun Du, and Jiajun Wu. 3d shape generation and completion through point-voxel diffusion. In *International Conference on Computer Vision (ICCV)*, 2021.
- [16] Shitong Luo and Wei Hu. Diffusion probabilistic models for 3d point cloud generation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [17] SAM 3D Team, Xingyu Chen, Fu-Jen Chu, Pierre Gleize, Kevin J Liang, Alexander Sax, Hao Tang, Weiyao Wang, Michelle Guo, Thibaut Hardin, Xiang Li, Aohan Lin, Jiawei Liu, Ziqi Ma, Anushka Sagar, Bowen Song, Xiaodong Wang, Jianing Yang, Bowen Zhang, Piotr Dollár, Georgia Gkioxari, Matt Feiszli, and Jitendra Malik. Sam 3d: 3dfy anything in images. 2025.
- [18] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020.
- [19] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)*, 42(4):139:1–139:14, 2023.
- [20] Yangguang Li, Zi-Xin Zou, Zexiang Liu, Dehu Wang, Yuan Liang, Zhipeng Yu, Xingchao Liu, Yuan-Chen Guo, Ding Liang, Wanli Ouyang, and Yan-Pei Cao. TripoSG: High-Fidelity 3D Shape Synthesis Using Large-Scale Rectified Flow Models. *IEEE Transactions on Pattern Analysis & Machine Intelligence*, (01):1–18, November 5555.
- [21] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. Structured 3d latents for scalable and versatile 3d generation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

- [22] Maria Parelli, Michael Oechsle, Michael Niemeyer, Federico Tombari, and Andreas Geiger. 3d-latte: Latent space 3d editing from textual instructions. *arXiv preprint arXiv:2509.00269*, 2025.
- [23] Haoxuan Li, Zeynep Erkoç, Lei Li, Daniele Sirigatti, Vladyslav Rosov, Angela Dai, and Matthias Nießner. Mesh-pad: Interactive sketch-conditioned artist-reminiscent mesh generation and editing. In *International Conference on Computer Vision (ICCV)*, 2025.
- [24] Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. Abc: A big cad model dataset for geometric deep learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9601–9611, 2019.
- [25] Ari Seff, Yaniv Ovadia, Wenda Zhou, and Ryan P. Adams. Sketchgraphs: A large-scale dataset for modeling relational geometry in computer-aided design. *arXiv preprint arXiv:2007.08506*, 2020.
- [26] Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences. *ACM Transactions on Graphics (TOG)*, 40(4):54:1–54:24, 2021.
- [27] Karl D. D. Willis, Pradeep Kumar Jayaraman, Hang Chu, Yunsheng Tian, Yifei Li, Daniele Grandi, Aditya Sanghi, Linh Tran, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Joinable: Learning bottom-up assembly of parametric cad joints. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [28] Elona Dupont, Kseniya Cherenkova, Anis Kacem, Sk Aziz Ali, Ilya Arzhannikov, Gleb Gusev, and Djamila Aouada. Cadops-net: Jointly learning cad operation types and steps from boundary representations. In *International Conference on 3D Vision (3DV)*, pages 114–123, 2022.
- [29] Hu Wang, Mingyuan Zhao, Yuxin Wang, Weize Quan, and Dong-Ming Yan. Vq-cad: Computer-aided design model generation with vector quantized diffusion. *Computer Aided Geometric Design*, 111:102327, 2024.
- [30] Mingi Lee, Dongsu Zhang, Clément Jambon, and Young Min Kim. Brepdiff: Single-stage b-rep diffusion model. In *ACM SIGGRAPH Conference Papers*, 2025.
- [31] Yilin Liu, Duoteng Xu, Xingguang Yu, Xiang Xu, Daniel Cohen-Or, Hao Zhang, and Hui Huang. Hola: B-rep generation using a holistic latent representation. *ACM Transactions on Graphics (Proceedings of SIGGRAPH)*, 44(4), 2025.
- [32] Xiang Xu, Pradeep Kumar Jayaraman, Joseph G. Lambourne, Yilin Liu, Dhruv Malpure, and Peter Meltzer. Autobrep: Autoregressive b-rep generation with unified topology and geometry. In *SIGGRAPH Asia Conference Papers*, 2025.
- [33] Pu Li, Wenhao Zhang, Weize Quan, Bo Zhang, Peter Wonka, and Dong-Ming Yan. Breppt: Autoregressive b-rep generation with voronoi half-patch. *ACM Transactions on Graphics (TOG)*, 44(6):1–18, 2025.
- [34] Yuan Li, Cheng Lin, Yuan Liu, Xiaoxiao Long, Chenxu Zhang, Ning Wang, Xin Li, Wenping Wang, and Xiaohu Guo. Caddreamer: Cad object generation from single-view images. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [35] Pradeep Kumar Jayaraman, Aditya Sanghi, Joseph G. Lambourne, Karl D. D. Willis, Thomas Davies, Hooman Shayani, and Nigel Morris. Uv-net: Learning from boundary representations. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11703–11712, 2021.
- [36] Yaroslav Ganin, Sergey Bartunov, Yujia Li, Ethan Keller, and Stefano Saliceti. Computer-aided design as language. *Advances in Neural Information Processing Systems*, 34, 2021.
- [37] Wamiq Para, Shariq Bhat, Paul Guerrero, Tom Kelly, Niloy J. Mitra, Leonidas J. Guibas, and Peter Wonka. Sketchgen: Generating constrained cad sketches. In *Advances in Neural Information Processing Systems*, volume 34, pages 5077–5088, 2021.
- [38] Charlie Nash, Yaroslav Ganin, S. M. Ali Eslami, and Peter Battaglia. Polygen: An autoregressive generative model of 3d meshes. In *International Conference on Machine Learning (ICML)*, pages 7220–7229, 2020.
- [39] Karl D. D. Willis, Pradeep Kumar Jayaraman, Joseph G. Lambourne, Hang Chu, and Yewen Pu. Engineering sketch generation for computer-aided design. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2105–2114, 2021.
- [40] Xiang Xu, Karl D. D. Willis, Joseph G. Lambourne, Chin-Yi Cheng, Pradeep Kumar Jayaraman, and Yasutaka Furukawa. Skexgen: Autoregressive generation of cad construction sequences with disentangled codebooks. In *International Conference on Machine Learning (ICML)*, pages 24698–24724, 2022.

- [41] Xiang Xu, Pradeep Kumar Jayaraman, Joseph G. Lambourne, Karl D. D. Willis, and Yasutaka Furukawa. Hierarchical neural coding for controllable cad model generation. In *International Conference on Machine Learning (ICML)*, pages 38443–38461, 2023.
- [42] Mohammad Sadil Khan, Elona Dupont, Sk Aziz Ali, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-signet: Cad language inference from point clouds using layer-wise sketch instance guided attention. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [43] Elona Dupont, Kseniya Cherenkova, Dimitrios Mallis, Gleb Gusev, Anis Kacem, and Djamila Aouada. Transcad: A hierarchical transformer for cad sequence inference from point clouds. In *European Conference on Computer Vision (ECCV)*, pages 19–36, 2024.
- [44] Weijian Ma, Minyang Xu, Xueyang Li, and Xiangdong Zhou. Multicad: Contrastive representation learning for multi-modal 3d computer-aided design models. In *ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1766–1776, 2023.
- [45] Ruiyu Wang, Yuyang Yuan, Shizhao Sun, and Jiang Bian. Text-to-cad generation through infusing visual feedback in large language models. *arXiv preprint arXiv:2501.19054*, 2025.
- [46] Sifan Wu, Amir Hosein Khasahmadi, Mor Katz, Pradeep Kumar Jayaraman, Yewen Pu, Karl D. D. Willis, and Bang Liu. Cadvlm: Bridging language and vision in the generation of parametric cad sketches. In *European Conference on Computer Vision (ECCV)*, 2024.
- [47] Cheng Chen, Jiacheng Wei, Tianrun Chen, Chi Zhang, Xiaofeng Yang, Shangzhan Zhang, Bingchen Yang, Chuan-Sheng Foo, Guosheng Lin, Qixing Huang, and Fayao Liu. Cadcrafter: Generating computer-aided design models from unconstrained images. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [48] Jiahao Li, Weijian Ma, Xueyang Li, Yunzhong Lou, Guichun Zhou, and Xiangdong Zhou. Cad-llama: Leveraging large language models for computer-aided design parametric 3d model generation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18563–18573, 2025.
- [49] Zhanwei Zhang, Shizhao Sun, Wenxiao Wang, Deng Cai, and Jiang Bian. Flexcad: Unified and versatile controllable cad generation with fine-tuned large language models. In *International Conference on Learning Representations (ICLR)*, 2025.
- [50] Jingwei Xu, Chenyu Wang, Zibo Zhao, Wen Liu, Yi Ma, and Shenghua Gao. Cad-mllm: Unifying multimodality-conditioned cad generation with mllm. *arXiv preprint arXiv:2411.04954*, 2024.
- [51] Siyu Wang, Cheng Chen, Xulin Le, Qingyu Xu, Lijun Xu, Yuxuan Zhang, and Jie Yang. Cad-gpt: Synthesising cad construction sequence with spatial reasoning-enhanced multimodal llms. In *AAAI Conference on Artificial Intelligence*, volume 39, pages 7880–7888, 2025.
- [52] Haoyang Xie and Feng Ju. Text-to-cadquery: A new paradigm for cad generation with scalable large model capabilities, 2025.
- [53] Akshay Badagabettu, Sai Sravan Yarlagadda, and Amir Barati Farimani. Query2cad: Generating cad models using natural language queries, 2024.
- [54] Zhuo Zuo, Yantao Gan, Junfeng Long, and Xianggen Liu. Cad-hllm: Generating executable cad from text with hierarchical llm planning. In Hung-yi Lee and Tongliang Liu, editors, *Proceedings of the 17th Asian Conference on Machine Learning*, volume 304 of *Proceedings of Machine Learning Research*, pages 958–973. PMLR, 09–12 Dec 2025.
- [55] Nomi Yu, Md Ferdous Alam, A. John Hart, and Faez Ahmed. Gencad-three-dimensional: Computer-aided design program generation using multimodal latent space alignment and synthetic dataset balancing. *Journal of Mechanical Design*, 148(3), 2025.
- [56] Mohammadmehdi Ataei, Farzaneh Askari, Kamal Rahimi Malekshan, and Pradeep Kumar Jayaraman. Zero-to-cad: Agentic synthesis of interpretable cad programs at million-scale without real data. *arXiv preprint arXiv:2604.24479*, 2026.
- [57] Jesse Barkley, Rumi Loghmani, and Amir Barati Farimani. Cadsmith: Multi-agent cad generation with programmatic geometric validation, 2026.
- [58] Yifei Gong, Xing Wu, Wenda Liu, and Kang Tu. Toolcad: Exploring tool-using large language models in text-to-cad generation with reinforcement learning, 2026.
- [59] Bharathi Kannan Nithyanantham, Tobias Sesterhenn, Ashwin Nedungadi, Sergio Peral Garijo, Janis Zenkner, Christian Bartelt, and Stefan Lüdtkke. Mcp4ifc: Ifc-based building design using large language models. *arXiv preprint arXiv:2511.05533*, 2025.

- [60] Zeqing Yuan, Haoxuan Lan, Qiang Zou, and Junbo Zhao. 3d-premise: Can large language models generate 3d shapes with sharp features and parametric control? *arXiv preprint arXiv:2401.06437*, 2024.
- [61] Zihan Deng, Changyu Du, Stavros Noutsias, and André Borrmann. Bimgent: Towards autonomous building modeling via computer-use agents. *arXiv preprint arXiv:2506.07217*, 2025.
- [62] Tao Hu, Jiaxin Ai, Licheng Wen, Xueheng Li, Shu Zou, Siqi Li, Nianchen Deng, Xinyu Cai, Hongbin Zhou, Pinlong Cai, Daocheng Fu, Yu Yang, Hairong Zhang, Botian Shi, and Xuemeng Yang. Itercad: An iterative multimodal agent for visually-grounded cad generation and editing, 2026.
- [63] Tong Wu, Guandao Yang, Zhibing Li, Kai Zhang, Ziwei Liu, Leonidas Guibas, Dahua Lin, and Gordon Wetzstein. Gpt-4v(ision) is a human-aligned evaluator for text-to-3d generation, 2024.
- [64] Yuewan Sun and Zhenghui Sha. Llm4cad-editor: An intent-aware large language model framework for multi-level computer-aided design editing, 2026.
- [65] Yunqi Gu, Ian Huang, Jihyeon Je, Guandao Yang, and Leonidas Guibas. Blendergym: Benchmarking foundational model systems for graphics editing, 2025.
- [66] William P. McCarthy, Saujas Vaduguru, Karl D. D. Willis, Justin Matejka, Judith E. Fan, Daniel Fried, and Yewen Pu. mrcad: Multimodal refinement of computer-aided designs, 2025.
- [67] Qwen Team, Alibaba Cloud. Qwen/qwen3.6-27b - hugging face. Hugging Face Model Hub, 2026.
- [68] Google DeepMind. google/gemma-4-31b - hugging face. Hugging Face Model Hub, 2026.
- [69] OpenAI via OpenRouter. Gpt-5.6 luna - api pricing & benchmarks | openrouter. OpenRouter Model Directory, 2026.
- [70] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2024.